
UNIT 9 PRIORITY QUEUES

Structure

- 9.0 Introduction
- 9.1 Objectives
- 9.2 The Priority Queue Abstract Data Type
- 9.3 Implementing a Priority Queue
- 9.4 Heaps
- 9.5 Sorting with a Priority Queue
- 9.6 Adaptable Priority Queues
- 9.7 Summary
- 9.8 Solutions/ Answer to Check your Progress
- 9.9 Further Readings

9.0 Introduction

Priority queues are fundamental data structures that extend the concept of regular queues by associating each element with a priority value. Priority queues use priority levels for managing the elements.

Priority queues find extensive applications in operating systems for process scheduling, network routing protocols, graph algorithms like Dijkstra's shortest path, and simulation systems. Priority queues can be implemented using heaps.

This unit explores the theoretical foundations and practical implementations of priority queues, with particular emphasis on heap-based implementations and their applications in sorting algorithms.

9.1 Objectives

After completing this unit, you will be able to:

- Understand the fundamental concepts and operations of priority queues
- Distinguish between different priority queue implementations and their trade-offs
- Implement priority queues using various data structures including heaps
- Master heap data structure properties and operations
- Apply heap-based algorithms for efficient sorting (Heap Sort)
- Implement adaptable priority queues with key modification capabilities
- Analyze time and space complexity of priority queue operations
- Apply priority queues to solve real-world problems in scheduling and optimization

9.2 The Priority Queue Abstract Data Type

A priority queue is an abstract data type that maintains a collection of elements, each associated with a priority value. The key characteristic is that elements are served in order of their priority rather than their insertion order.

Key Properties:

- Each element has an associated priority
- Higher priority elements are served before lower priority elements
- Elements with equal priority may be served in any order (or FIFO order in some implementations)

PRIORITY QUEUE OPERATIONS

The essential operations for a priority queue include:

Primary Operations:

- `insert(item, priority)`: Add an item with given priority
- `extract_max()` or `extract_min()`: Remove and return highest/lowest priority item
- `peek()`: Return highest/lowest priority item without removing it
- `is_empty()`: Check if the priority queue is empty
- `size()`: Return the number of elements

Additional Operations:

- `change_priority(item, new_priority)`: Modify the priority of an existing item
- `remove(item)`: Remove a specific item from the queue

APPLICATIONS OF PRIORITY QUEUES

Priority queues have numerous real-world applications:

Operating Systems:

- Process scheduling based on priority levels
- Memory management for page replacement algorithms

Network Management:

- Packet routing with Quality of Service (QoS) requirements
- Bandwidth allocation for different traffic types

Graph Algorithms:

- Dijkstra's shortest path algorithm
- Prim's minimum spanning tree algorithm
- A* search algorithm

Simulation Systems:

- Event-driven simulations where events have timestamps
- Task scheduling in project management

Data Processing:

- Top-K problems (finding K largest/smallest elements)
- Merge operations in external sorting

9.3 Implementing a Priority Queue

ARRAY-BASED IMPLEMENTATION

The simplest approach uses an unsorted array to store priority-item pairs:

```

class ArrayPriorityQueue:
    """Simple array-based priority queue implementation."""

    def __init__(self):
self._data = []

    def insert(self, item, priority):
    """Insert item with priority. O(1) time complexity."""
self._data.append((priority, item))

    def extract_min(self):
    """Remove and return minimum priority item. O(n) time complexity."""
    if self.is_empty():
        raise ValueError("Priority queue is empty")

# Find minimum priority item
min_index = 0
    for i in range(1, len(self._data)):
        if self._data[i][0] < self._data[min_index][0]:
            min_index = i

# Remove and return the item
    priority, item = self._data.pop(min_index)
    return item

    def peek_min(self):
    """Return minimum priority item without removing. O(n) time
complexity."""
    if self.is_empty():
        raise ValueError("Priority queue is empty")

min_priority = float('inf')
min_item = None
    for priority, item in self._data:
        if priority < min_priority:
            min_priority = priority
            min_item = item
    return min_item

    def is_empty(self):
    """Check if queue is empty. O(1) time complexity."""
    return len(self._data) == 0

    def size(self):
    """Return number of items. O(1) time complexity."""
    return len(self._data)

# Example usage
pq = ArrayPriorityQueue()
pq.insert("Task A", 3)
pq.insert("Task B", 1)
pq.insert("Task C", 2)

print(pq.extract_min()) # Output: Task B (priority 1)

```

```
print(pq.peek_min()) # Output: Task C (priority 2)
```

SORTED LIST IMPLEMENTATION

An alternative approach maintains elements in sorted order:

python

```
class SortedListPriorityQueue:
    """Priority queue using sorted list implementation."""

    def __init__(self):
        self._data = [] # Maintained in sorted order by priority

    def insert(self, item, priority):
        """Insert item maintaining sorted order. O(n) time complexity."""
        # Find correct position using binary search
        import bisect
        bisect.insort(self._data, (priority, item))

    def extract_min(self):
        """Remove and return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")

        priority, item = self._data.pop(0)
        return item

    def peek_min(self):
        """Return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")
        return self._data[0][1]

    def is_empty(self):
        return len(self._data) == 0

    def size(self):
        return len(self._data)
```

LINKED LIST IMPLEMENTATION

Using linked lists provides flexibility for dynamic sizing:

python

```
class Node:
    """Node class for linked list implementation."""
    def __init__(self, item, priority):
        self.item = item
        self.priority = priority
        self.next = None

class LinkedListPriorityQueue:
    """Priority queue using sorted linked list."""

    def __init__(self):
        self._head = None
        self._size = 0

    def insert(self, item, priority):
```

```

        """Insert item maintaining priority order. O(n) time complexity."""
        new_node = Node(item, priority)

        # Insert at beginning if empty or highest priority
        if not self._head or priority < self._head.priority:
            new_node.next = self._head
            self._head = new_node
        else:
            # Find correct position
            current = self._head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node

        self._size += 1

    def extract_min(self):
        """Remove and return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")

        item = self._head.item
        self._head = self._head.next
        self._size -= 1
        return item

    def peek_min(self):
        """Return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")
        return self._head.item

    def is_empty(self):
        return self._head is None

    def size(self):
        return self._size

```

COMPARISON OF IMPLEMENTATIONS

Implementation	Insert	Extract Min	Peek Min	Space
Unsorted Array	O(1)	O(n)	O(n)	O(n)
Sorted Array	O(n)	O(1)	O(1)	O(n)
Sorted Linked List	O(n)	O(1)	O(1)	O(n)
Binary Heap	O(log n)	O(log n)	O(1)	O(n)

The binary heap implementation provides the best overall performance, which we'll explore in the next section.

9.4 Heaps

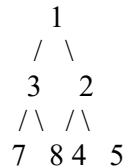
A heap uses a specialized tree-based data structure that satisfies the heap property. It's the most effective way to implement a priority queue.

Heap Properties:

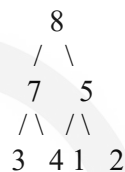
1. **Structure Property:** A heap is a complete binary tree (all levels filled except possibly the last, which is filled left-to-right)
2. **Order Property:** The priority of each node is greater than or equal to (max-heap) or less than or equal to (min-heap) its children's priorities

TYPES OF HEAPS

1. **Min-Heap:** Parent node has smaller priority than its children



2. **Max-Heap:** Parent node has larger priority than its children



HEAP REPRESENTATION

3. Heaps are typically implemented using arrays, providing efficient storage and easy navigation:

```

class MinHeap:
    """Array-based min-heap implementation."""

    def __init__(self):
        self._data = []

    def _parent_index(self, index):
        """Return parent index of given index."""
        return (index - 1) // 2

    def _left_child_index(self, index):
        """Return left child index."""
        return 2 * index + 1

    def _right_child_index(self, index):
        """Return right child index."""
        return 2 * index + 2

    def _has_parent(self, index):
        """Check if node has parent."""
        return self._parent_index(index) >= 0

    def _has_left_child(self, index):
        """Check if node has left child."""
        return self._left_child_index(index) < len(self._data)

```

```

def _has_right_child(self, index):
    """Check if node has right child."""
    return self._right_child_index(index) < len(self._data)

def _parent(self, index):
    """Get parent element."""
    return self._data[self._parent_index(index)]

def _left_child(self, index):
    """Get left child element."""
    return self._data[self._left_child_index(index)]

def _right_child(self, index):
    """Get right child element."""
    return self._data[self._right_child_index(index)]

def _swap(self, index1, index2):
    """Swap elements at two indices."""
    self._data[index1], self._data[index2] = self._data[index2], self._data[index1]

```

HEAP OPERATIONS

Insertion (Heapify Up): Insert at the end and bubble up to maintain heap property.

```

def insert(self, item, priority):
    """Insert item with priority. O(log n) time complexity."""
    self._data.append((priority, item))
    self._heapify_up(len(self._data) - 1)

def _heapify_up(self, index):
    """Restore heap property by moving element up."""
    while (self._has_parent(index) and
           self._parent(index)[0] > self._data[index][0]):
        parent_idx = self._parent_index(index)
        self._swap(index, parent_idx)
        index = parent_idx

```

Extraction (Heapify Down): Remove root, replace with last element, and bubble down.

```

def extract_min(self):
    """Remove and return minimum element. O(log n) time complexity."""
    if self.is_empty():
        raise ValueError("Heap is empty")

    # Store minimum element
    min_item = self._data[0][1]

    # Replace root with last element
    self._data[0] = self._data[-1]
    self._data.pop()

    # Restore heap property
    if not self.is_empty():

```

```

self._heapify_down(0)

return min_item

def _heapify_down(self, index):
    """Restore heap property by moving element down."""
    while self._has_left_child(index):
        # Find smaller child
        smaller_child_idx = self._left_child_index(index)
        if (self._has_right_child(index) and
            self._right_child(index)[0] < self._left_child(index)[0]):
            smaller_child_idx = self._right_child_index(index)

        # Check if swap is needed
        if self._data[index][0] < self._data[smaller_child_idx][0]:
            break

        # Swap and continue
        self._swap(index, smaller_child_idx)
        index = smaller_child_idx

```

Complete Min-Heap Implementation:

```

class MinHeapPriorityQueue:
    """Complete min-heap based priority queue implementation."""

    def __init__(self):
        self._data = []

    def _parent_index(self, index):
        return (index - 1) // 2

    def _left_child_index(self, index):
        return 2 * index + 1

    def _right_child_index(self, index):
        return 2 * index + 2

    def _has_parent(self, index):
        return self._parent_index(index) >= 0

    def _has_left_child(self, index):
        return self._left_child_index(index) < len(self._data)

    def _has_right_child(self, index):
        return self._right_child_index(index) < len(self._data)

    def _parent(self, index):
        return self._data[self._parent_index(index)]

    def _left_child(self, index):
        return self._data[self._left_child_index(index)]

    def _right_child(self, index):

```

```

    return self._data[self._right_child_index(index)]

    def _swap(self, index1, index2):
self._data[index1], self._data[index2] = self._data[index2], self._data[index1]

    def insert(self, item, priority):
        """Insert item with priority. O(log n) time complexity."""
self._data.append((priority, item))
self._heapify_up(len(self._data) - 1)

    def _heapify_up(self, index):
        """Restore heap property by moving element up."""
        while (self._has_parent(index) and
self._parent(index)[0] > self._data[index][0]):
parent_idx = self._parent_index(index)
self._swap(index, parent_idx)
        index = parent_idx

    def extract_min(self):
        """Remove and return minimum element. O(log n) time complexity."""
        if self.is_empty():
            raise ValueError("Heap is empty")

min_item = self._data[0][1]
self._data[0] = self._data[-1]
self._data.pop()

        if not self.is_empty():
self._heapify_down(0)

        return min_item

    def _heapify_down(self, index):
        """Restore heap property by moving element down."""
        while self._has_left_child(index):
smaller_child_idx = self._left_child_index(index)
            if (self._has_right_child(index) and
self._right_child(index)[0] < self._left_child(index)[0]):
smaller_child_idx = self._right_child_index(index)

            if self._data[index][0] < self._data[smaller_child_idx][0]:
                break

self._swap(index, smaller_child_idx)
        index = smaller_child_idx

    def peek_min(self):
        """Return minimum element without removing. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Heap is empty")
        return self._data[0][1]

    def is_empty(self):
        """Check if heap is empty. O(1) time complexity."""

```

```

        return len(self._data) == 0

    def size(self):
        """Return number of elements. O(1) time complexity."""
        return len(self._data)

# Example usage and demonstration
    def demonstrate_heap():
        """Demonstrate heap operations."""
        heap = MinHeapPriorityQueue()

# Insert elements
        tasks = [("High Priority Task", 1), ("Medium Task 1", 3),
                  ("Low Priority Task", 5), ("Medium Task 2", 3),
                  ("Critical Task", 0)]

        print("Inserting tasks:")
        for task, priority in tasks:
            heap.insert(task, priority)
        print(f"Inserted: {task} (Priority: {priority})")

        print(f"\nHeap size: {heap.size()}")
        print(f"Next task: {heap.peek_min()}")

        print("\nProcessing tasks in priority order:")
        while not heap.is_empty():
            task = heap.extract_min()
            print(f"Processing: {task}")

# Run demonstration
    demonstrate_heap()

```

BUILDING A HEAP FROM ARRAY

We can build a heap from an existing array using bottom-up heapify:

```

def build_heap(self, array_of_pairs):
    """Build heap from array of (priority, item) pairs. O(n) time complexity."""
    self._data = array_of_pairs.copy()

# Start from last non-leaf node and heapify down
    start_index = (len(self._data) - 2) // 2
    for i in range(start_index, -1, -1):
        self._heapify_down(i)

# Example usage
    heap = MinHeapPriorityQueue()
    data = [(4, "Task D"), (1, "Task A"), (3, "Task C"), (2, "Task B"), (5, "Task E")]
    heap.build_heap(data)

    print("Heap after building from array:")
    while not heap.is_empty():
        print(heap.extract_min())

```

9.5 Sorting with a Priority Queue

Heap sort uses a priority queue (heap) to sort elements efficiently. It has $O(n \log n)$ time complexity and sorts in-place.

Algorithm Steps:

1. Build a max-heap from the input array
2. Repeatedly extract the maximum element and place it at the end
3. Reduce the heap size and restore heap property

```
def heap_sort(arr):
    """Sort array using heap sort algorithm.  $O(n \log n)$  time complexity."""

    def heapify(arr, n, root):
        """Maintain max-heap property."""
        largest = root
        left = 2 * root + 1
        right = 2 * root + 2

        # Find largest among root and children
        if left < n and arr[left] > arr[largest]:
            largest = left
        if right < n and arr[right] > arr[largest]:
            largest = right

        # If largest is not root, swap and continue heapifying
        if largest != root:
            arr[root], arr[largest] = arr[largest], arr[root]
            heapify(arr, n, largest)

    n = len(arr)

    # Build max-heap (bottom-up)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        # Move current root to end
        arr[0], arr[i] = arr[i], arr[0]

        # Restore heap property for reduced heap
        heapify(arr, i, 0)

    return arr

# Example usage
def demonstrate_heap_sort():
    """Demonstrate heap sort algorithm."""
    arrays = [
        [64, 34, 25, 12, 22, 11, 90],
        [5, 2, 8, 1, 9],
        [1],
        []
    ]
```

```

    ]

    for arr in arrays:
        original = arr.copy()
    sorted_arr = heap_sort(arr)
    print(f"Original: {original}")
    print(f"Sorted: {sorted_arr}")
    print()

    demonstrate_heap_sort()

```

PRIORITY QUEUE SORT

We can also sort using a priority queue data structure:

```

def priority_queue_sort(arr):
    """Sort array using priority queue. O(n log n) time complexity."""
    pq = MinHeapPriorityQueue()

    # Insert all elements
    for i, value in enumerate(arr):
        pq.insert(value, value)

    # Extract elements in sorted order
    sorted_arr = []
    while not pq.is_empty():
        sorted_arr.append(pq.extract_min())

    return sorted_arr

# Example usage
original = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = priority_queue_sort(original)
print(f"Original: {original}")
print(f"Sorted: {sorted_arr}")

```

TOP-K ELEMENTS PROBLEM

Finding the K largest or smallest elements is a common application:

```

def find_k_largest(arr, k):
    """Find K largest elements using min-heap. O(n log k) time complexity."""
    if k <= 0 or k > len(arr):
        return []

    # Use min-heap of size k
    import heapq
    min_heap = []

    for num in arr:
        if len(min_heap) < k:
            heapq.heappush(min_heap, num)
        elif num > min_heap[0]:
            heapq.heapreplace(min_heap, num)

    return sorted(min_heap, reverse=True)

```

```

def find_k_smallest(arr, k):
    """Find K smallest elements using max-heap. O(n log k) time complexity."""
    if k <= 0 or k > len(arr):
        return []

    # Use max-heap of size k (negate values for max-heap with min-heap)
    import heapq
    max_heap = []

    for num in arr:
        if len(max_heap) < k:
            heapq.heappush(max_heap, -num)
        elif num < -max_heap[0]:
            heapq.heapreplace(max_heap, -num)

    return sorted([-x for x in max_heap])

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90, 88, 76, 50]
print(f'Array: {arr}')
print(f'3 largest: {find_k_largest(arr, 3)}')
print(f'3 smallest: {find_k_smallest(arr, 3)}')

```

9.6 Adaptable Priority Queues

CONCEPT AND MOTIVATION

Adaptable priority queues support additional operations to modify existing entries:

- Change the priority of an existing item
- Remove a specific item from the queue

These operations are essential in applications like Dijkstra's algorithm where priorities need to be updated dynamically.

IMPLEMENTATION WITH LOCATION-AWARE ENTRIES

```

class Entry:
    """Entry class for adaptable priority queue."""
    def __init__(self, key, value, index):
        self._key = key    # priority
        self._value = value # item
        self._index = index # current index in heap

    def __lt__(self, other):
        return self._key < other._key

class AdaptablePriorityQueue:
    """Adaptable priority queue using binary heap."""

    def __init__(self):
        self._data = []
        self._entries = {} # Maps values to entries for fast lookup

```

```

def _parent(self, j):
    return (j - 1) // 2

def _left(self, j):
    return 2 * j + 1

def _right(self, j):
    return 2 * j + 2

def _has_left(self, j):
    return self._left(j) < len(self._data)

def _has_right(self, j):
    return self._right(j) < len(self._data)

def _swap(self, i, j):
    """Swap entries and update their indices."""
    self._data[i], self._data[j] = self._data[j], self._data[i]
    self._data[i]._index = i
    self._data[j]._index = j

def _upheap(self, j):
    """Restore heap property by moving entry up."""
    parent = self._parent(j)
    if j > 0 and self._data[j] < self._data[parent]:
        self._swap(j, parent)
        self._upheap(parent)

def _downheap(self, j):
    """Restore heap property by moving entry down."""
    if self._has_left(j):
        left = self._left(j)
        small_child = left
        if self._has_right(j):
            right = self._right(j)
            if self._data[right] < self._data[left]:
                small_child = right
        if self._data[small_child] < self._data[j]:
            self._swap(j, small_child)
            self._downheap(small_child)

def insert(self, key, value):
    """Insert new entry. O(log n) time complexity."""
    if value in self._entries:
        raise ValueError(f"Value {value} already exists")

    entry = Entry(key, value, len(self._data))
    self._data.append(entry)
    self._entries[value] = entry
    self._upheap(len(self._data) - 1)
    return entry

def min(self):

```

```

        """Return minimum entry without removing. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")
        return (self._data[0]._key, self._data[0]._value)

    def remove_min(self):
        """Remove and return minimum entry. O(log n) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")

    # Store minimum entry
    min_entry = self._data[0]
    min_key, min_value = min_entry._key, min_entry._value

    # Replace with last entry
    last_entry = self._data[-1]
    self._data[0] = last_entry
    last_entry._index = 0
    self._data.pop()

    # Remove from entries map
    del self._entries[min_value]

    # Restore heap property
    if not self.is_empty():
        self._downheap(0)

    return (min_key, min_value)

    def update(self, value, new_key):
        """Update priority of existing entry. O(log n) time complexity."""
        if value not in self._entries:
            raise ValueError(f"Value {value} not found")

        entry = self._entries[value]
        old_key = entry._key
        entry._key = new_key

    # Restore heap property
    if new_key < old_key:
        self._upheap(entry._index)
    elif new_key > old_key:
        self._downheap(entry._index)

    def remove(self, value):
        """Remove specific entry. O(log n) time complexity."""
        if value not in self._entries:
            raise ValueError(f"Value {value} not found")

        entry = self._entries[value]
        j = entry._index

        if j == len(self._data) - 1:
            # Last element, just remove

```

```

self._data.pop()
    else:
# Replace with last element
    last_entry = self._data[-1]
    self._data[j] = last_entry
    last_entry._index = j
    self._data.pop()

# Restore heap property
    self._upheap(j)
    self._downheap(j)

# Remove from entries map
    del self._entries[value]
    return (entry._key, value)

def is_empty(self):
    return len(self._data) == 0

def size(self):
    return len(self._data)

# Example usage and demonstration
def demonstrate_adaptable_pq():
    """Demonstrate adaptable priority queue operations."""
    pq = AdaptablePriorityQueue()

    # Insert tasks
    tasks = [("Task A", 3), ("Task B", 1), ("Task C", 4), ("Task D", 2)]

    print("Inserting tasks:")
    for task, priority in tasks:
        pq.insert(priority, task)
    print(f"Inserted {task} with priority {priority}")

    print(f"\nCurrent minimum: {pq.min()}")

    # Update priority
    print("\nUpdating Task C priority from 4 to 0:")
    pq.update("Task C", 0)
    print(f"New minimum: {pq.min()}")

    # Remove specific task
    print("\nRemoving Task A:")
    removed = pq.remove("Task A")
    print(f"Removed: {removed}")

    print("\nProcessing remaining tasks:")
    while not pq.is_empty():
        task = pq.

```

1. Question: Which of the following is NOT a type of tree traversal? a) Pre-order traversal b) In-order traversal c) Sequential traversal d) Post-order traversal
a. Answer: c) Sequential traversal
2. Question: Give the highest possible nodes at level 3 (assuming root is at level 0)? a) 4 b) 8 c) 16 d) 2
a. Answer: b) 8
3. Question: A binary tree with 10 nodes has how many NULL links (empty child references)? a) 9 b) 10 c) 11 d) 12
a. Answer: c) 11
4. Question: Which type of binary tree has all internal nodes containing exactly two children? a) Complete binary tree b) Full binary tree c) Perfect binary tree d) Balanced binary tree Answer: b) Full binary tree
5. Question: What is the order of visiting nodes in a pre-order traversal? a) Root, Right, Left b) Left, Root, Right c) Root, Left, Right d) Left, Right, Root
a. Answer: c) Root, Left, Right
6. Question: In an expression tree, where are the operands located? a) Only at the root b) Only at leaf nodes c) At any internal node d) Only at the second level
a. Answer: b)
7. Question: What would be the level-order traversal of the following binary tree?
Copy
1
 /\n 2 3
 /\n 4 5
a) 1, 2, 3, 4, 5 b) 1, 3, 2, 5, 4 c) 4, 2, 5, 1, 3 d) 1, 2, 3, 5, 4
a. Answer: a) 1, 2, 3, 4, 5
8. Question: A complete binary tree with 7 nodes has which height? a) 2 b) 3 c) 4 d) 5
a. Answer: a) 2
9. Question: Which data structure is typically used to implement level-order traversal? a) Stack b) Queue c) Linked List d) Array
a. Answer: b) Queue
10. Question: What is the time complexity of inserting a node into a binary tree using level-order insertion? a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$
a. Answer: c) $O(n)$

9.7 Summary

- Priority queues are fundamental data structures that extend regular queues by serving elements based on their priority rather than insertion order. This unit covered the essential aspects of priority queues and their most efficient implementation using heaps.

Key Concepts Covered:

- **Priority Queue Abstract Data Type:** Understanding the basic operations (insert, extract_min/max, peek) and their applications in scheduling, graph algorithms, and simulation systems.

- **Implementation Strategies:** We explored various approaches including unsorted arrays, sorted arrays, linked lists, and binary heaps, analyzing their time complexity trade-offs.
- **Binary Heaps:** The most efficient priority queue implementation, providing $O(\log n)$ insertion and extraction with $O(1)$ peek operations. We covered both min-heaps and max-heaps, their structural and ordering properties.
- **Heap Operations:** Detailed implementation of heapify-up and heapify-down operations, array-based representation, and bottom-up heap construction.
- **Sorting Applications:** Heap sort algorithm demonstrating how priority queues can achieve $O(n \log n)$ sorting performance, along with solutions to top-K problems.
- **Adaptable Priority Queues:** Advanced implementations supporting priority updates and specific element removal, essential for dynamic algorithms like Dijkstra's shortest path.

Performance Summary:

- Standard heap operations: $O(\log n)$ for insertion and extraction
- Heap construction from array: $O(n)$ time complexity
- Heap sort: $O(n \log n)$ time, $O(1)$ extra space
- Adaptable operations: $O(\log n)$ for updates and removals
- Priority queues find extensive applications in operating systems, network protocols, graph algorithms, and data processing tasks. Mastering these concepts provides a solid foundation for advanced algorithms and system design.

9.8 Solutions / Answers to Check Your Progress

1. Question: Which of the following is NOT a standard priority queue operation?
a) insert(item, priority) b) extract_min() c) sort_all() d) peek()

Answer: c) sort_all()

2. Question: In a min-heap with 15 elements, what is the maximum number of comparisons needed to insert a new element? a) 3 b) 4 c) 5 d) 15

Answer: b) 4

3. Question: Which array represents a valid max-heap? a) [90, 80, 70, 60, 50, 40, 30] b) [90, 70, 80, 60, 50, 30, 40] c) [90, 80, 85, 60, 70, 40, 30] d) [30, 40, 50, 60, 70, 80, 90]

Answer: c) [90, 80, 85, 60, 70, 40, 30]

4. Question: What is the time complexity of building a heap from an unsorted array using the bottom-up approach? a) $O(n \log n)$ b) $O(n^2)$ c) $O(n)$ d) $O(\log n)$

Answer: c) $O(n)$

•

5. Question: In heap sort, after building the initial max-heap, what operation is repeatedly performed? a) Insert the maximum element b) Extract maximum and heapify c) Search for minimum element d) Rebuild the entire heap

Answer: b) Extract maximum and heapify

6. Question: Which implementation provides the best overall time complexity for priority queue operations? a) Unsorted array b) Sorted array c) Binary heap d) Sorted linked list

- **Answer:** c) Binary heap

-

7. Question: In an adaptable priority queue, what additional capability is provided? a) Faster insertion b) Multiple priority levels c) Modifying existing entries d) Automatic sorting

Answer: c) Modifying existing entries

8. Question: What is the space complexity of the heap sort algorithm? a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n \log n)$

Answer: c) $O(1)$

9. Question: In a binary heap stored as an array, if a node is at index i , where is its left child? a) $2i$ b) $2i + 1$ c) $i + 1$ d) $i/2$

Answer: b) $2i + 1$

-

10. Question: Which of the following applications would benefit most from using a priority queue? a) FIFO task processing b) CPU process scheduling c) Stack-based calculations d) Sequential data storage

Answer: b) CPU process scheduling

-

9.9 Further Readings

<https://www.geeksforgeeks.org/dsa/priority-queue-set-1-introduction/>